CSC490 Assignment A4 SFT Nanochat

Ananya Jain (1007706573), **Aviral Bhardwaj** (1008239407)
Ansh Agrawal (1007413103), **Mann Thakkar** (1007647744)

1 Part One: GRPO and RL Review

Feature	**Standard GRPO** [4]	**Nanochat**	**Why This Change**
KL Divergence	Includes $-\beta D_{KL}(\pi_\theta \| \pi_{ref})$	Removed	Avoids loading a reference model, reducing VRAM use.
Advantage	$\frac{r_i - \text{mean}(r)}{\text{std}(r)}$	$r_i - \text{mean}(r)$	More stable at small sample sizes.
Trust Region	PPO-style clipping term	No clipping	Single-pass on-policy updates make clipping less necessary.
Granularity	Sequence-level rewards	Token-level rewards	Better credit assignment; length-normalized token loss.

Table 1: Comparison of standard GRPO and Nanochat's RL implementation.

In Karpathy's Nanochat implementation (`scripts/chat_rl.py`), training uses REINFORCE rather than full GRPO. A core design goal is to optimize wall-clock efficiency and memory usage. By removing both the reference model (for KL regularization) and the critic model (for value estimation), Nanochat can train 1B+ parameter models on a single consumer GPU.

In addition, hyperparameters such as KL-α, KL-β, and clipping-ϵ are often difficult to tune. Removing these leaves one primary control variable, the learning rate, which is simpler to tune in practice. For GSM8K, the ground-truth integer answer provides a strong supervision signal. While standard GRPO smooths updates, for this reasoning-heavy setting, the direct REINFORCE objective is often sufficient to push the policy toward correct reasoning chains.

2 Part Two: SFT & Midtraining

2.1 Baseline SFT and Midtraining

We ran the SFT script on a freshly pretrained nanochat (depth=20) using Karpathy's most recent repository updates rather than our A3 model, as these upstream changes represent a stronger and more current baseline for SFT and RL experiments. All runs

were logged to Weights & Biases using the original default configuration. The pretrained model was evaluated on the CORE benchmark suite, while the SFT model is evaluated on ChatCORE. Table 2 summarizes the key metrics, and Table 3 shows the task-level breakdown for both models.

Metric	**Pretrained**	**SFT**
Model Step	4357	485
Val BPB ↓	0.7413	**0.2924**
Overall Score ↑	0.2267 (CORE)	**0.3226** (ChatCORE)

Table 2: High-level comparison between pretrained and SFT model.

Task	**Pretrained (CORE centered)**	**SFT (Accuracy)**
ARC Easy	0.5533	0.5636
ARC Challenge	0.1684	**0.4539**
Hellaswag (0-shot)	0.3369	—
PIQA	0.4407	—
WinoGrad	0.2894	—
LaMBADA	0.4046	—
MMLU	—	0.3426
GSM8K	—	0.0629
HumanEval	—	0.0976
Spelling Bee	—	1.0000

Table 3: Task-level breakdown comparing pretrained and SFT models.

The SFT model shows a large drop in val BPB from 0.7413 to 0.2924, which is expected as SFT data consists of structured instruction-response pairs that are much more predictable in format than web-scale pretraining text.

On tasks where both models can be compared, ARC Challenge improves substantially from 0.1684 to 0.4539, suggesting that instruction tuning helped the model better understand how to approach multiple-choice questions. ARC Easy stays roughly the same, which makes sense as the pretrained model was already performing well on it.

For SFT the weakest results are on GSM8K (0.0629) and HumanEval (0.0976), which require multi-step reasoning rather than just format understanding. The model has learned to follow instructions but has not yet developed reliable mathematical reasoning. Spelling Bee reaching a perfect 1.0 shows the model picked up simple rule-following tasks well from the SFT data.

Overall, SFT meaningfully improved the model's instruction-following ability and overall ChatCORE score from 0.2267 to 0.3226, but the low GSM8K score confirms that SFT alone is not enough to teach mathematical reasoning.

2.2 Additional SFT Dataset Runs

To improve on the original SFT baseline, we experimented with adding math and code-focused datasets to the training mixture. Our goal was to address the two clearest

weaknesses in the original SFT model: low GSM8K accuracy (0.0629) and low HumanEval accuracy (0.0976).

Dataset Choices

We selected two additional datasets based on literature support:

MetaMath (Yu et al. [1]) rephrases existing GSM8K and MATH problems into new variants, creating a dataset of 395K math instruction pairs. The paper shows consistent GSM8K improvements across model sizes, making it a well-motivated choice for our primary weakness.

Magicoder (Wei et al. [2]) generates 75K code instruction pairs from real open source code snippets using OSS-Instruct. Unlike self-instruct methods such as CodeAlpaca, Magicoder uses actual code as seed data which produces higher quality and more diverse examples. Beyond directly targeting HumanEval, Gao et al. [3] show that code-trained models tend to reason better mathematically since both tasks require sequential logical steps, motivating its inclusion alongside MetaMath.

Datamix

Both additional runs used the same base configuration as the original SFT, with MMLU and GSM8K epochs reduced to x2 to keep training time comparable, and new datasets added at x2 epochs each.

Metric	Original SFT	MetaMath	Magicoder + MetaMath
Model Step	485	836	708
Val BPB ↓	0.2924	0.2921	**0.2905**
ChatCORE ↑	0.3226	0.3493	**0.3681**
ARC Easy	0.5636	0.5459	**0.5699**
ARC Challenge	0.4539	0.4394	**0.4497**
MMLU	0.3426	0.3427	**0.3557**
GSM8K	0.0629	**0.2274**	0.2083
HumanEval	0.0976	0.0732	**0.1667**
Spelling Bee	1.0000	1.0000	1.0000

Table 4: Comparison of original SFT, MetaMath, and Magicoder+MetaMath runs.

Adding MetaMath to the training mixture produced a large improvement on GSM8K, jumping from 0.0629 to 0.2274, a 3.6× increase over the original SFT baseline. This confirmed that the original SFT data did not provide enough math-specific examples for the model to develop reliable arithmetic reasoning. The Magicoder+MetaMath run was our strongest overall result, achieving the best ChatCORE (0.3681), best val BPB (0.2905), and best HumanEval (0.1667). HumanEval nearly doubled from the original SFT baseline, which we attribute directly to Magicoder's higher quality code examples derived from real open source snippets.

The only metric where MetaMath alone outperforms Magicoder+MetaMath is GSM8K (0.2274 vs 0.2083). Splitting the training budget between math and code data reduces total math-focused exposure, producing a small GSM8K tradeoff in exchange for broader gains across MMLU, ARC, and HumanEval.

3 Part Three: RL Replication and Analysis

3.1 GSM8k Training Replication

Figure 1: reward curve, eval pass@1, and eval pass@8; top row: our RL run; bottom row: nanochat original curve.

Figure 1 compares our reinforcement learning training run to the results reported by Karpathy in the nanochat RL implementation.

Overall, both runs show that RL improves performance over the supervised baseline. In our run, **Pass@1** increased from approximately 4.5% to 11.5%, **Pass@2** increased from about 7% to 18%, and **Pass@8** increased from about 22.5% to 28%. This trend is consistent with the original run, where evaluation accuracy also improves during RL training. As in the original experiment, **Pass@8** remains substantially higher than **Pass@1**, indicating that the model can often generate a correct answer among multiple samples but still struggles to produce it consistently in a single attempt.

A key difference is the **reward-curve behavior**. In the original run, reward generally trends upward, indicating stable policy improvement. In contrast, our recent reward curve is highly noisy and shows no clear upward trend, with large fluctuations across training steps. This is likely due to high variance in GSM8K problem difficulty across batches, small batch sizes, and the simplified RL setup that removes stabilizing techniques such as KL penalties and PPO clipping.

Our evaluation curves also show higher volatility, including an early spike around step $\sim$ 100 followed by a drop before stabilizing and improving later in training. This may result from lucky early batches, a relatively high learning rate, or unstable policy updates.

3.2 Problem Analysis and Exploratory Data Analysis

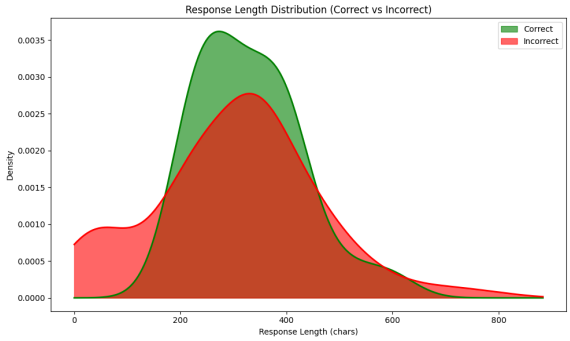

Figure 2: Response length for correct vs incorrect responses.

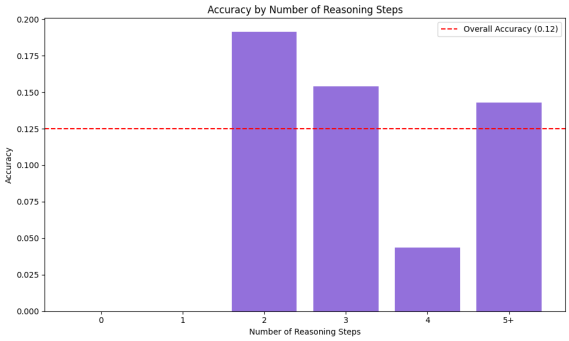

Figure 3: Accuracy by detected reasoning steps.

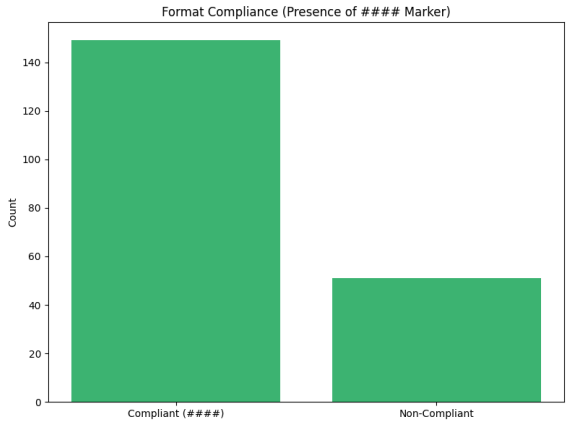

Figure 4: Format compliance showing presence of #### marker across all responses.

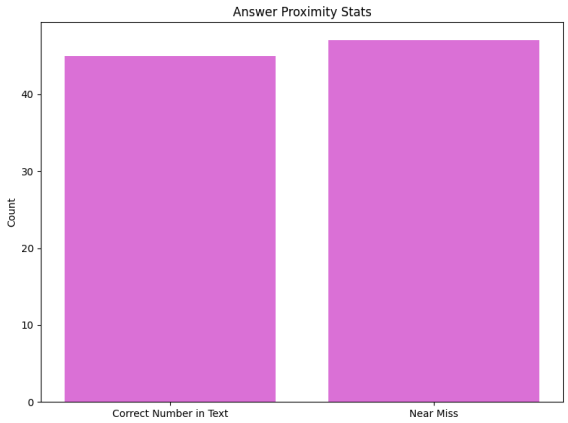

Figure 5: Answer proximity stats for incorrect responses

Response Length Analysis

Figure 2 shows that correct responses are more tightly concentrated between 250 and 400 characters, while incorrect responses have a notable bump near 0–100 characters and a longer right tail extending beyond 600 characters. The short incorrect responses represent cases where the model produced near-empty or malformed outputs, while the long tail suggests the model sometimes rambles when it cannot find a solution.

Reasoning Steps Analysis

Figure 3 shows that responses with 0 or 1 detected reasoning steps have zero accuracy, while 2-step responses achieve the highest accuracy at 0.19. Accuracy then drops at 4 steps to 0.04, suggesting that longer reasoning chains introduce more opportunities for compounding errors rather than improving correctness.

Format Compliance

Figure 4 shows that approximately 149 out of 200 responses included the #### answer marker, meaning around 25% of responses were non-compliant. Non-compliant responses

are almost always incorrect since the answer cannot be extracted without the marker, making format compliance a clear target for an additional reward signal in Part 4.

Answer Proximity

Figure 5 shows that among incorrect responses, approximately 45 contained the correct answer number somewhere in the generated text and 47 were near misses. Together these account for a substantial fraction of failures, indicating the model frequently reasons in the right direction but fails to isolate the correct final answer. This motivates the proximity reward in Part 4, which provides partial credit when the correct number appears in the response.

4 Part Four: Complex Rewards System

4.1 Additional Reward Definitions

Motivated by Part Three error patterns (high truncation and frequent near misses), we kept the baseline GSM8K exact-match reward and added two shaped rewards. The combined reward can be written as:

$$r_{total} = r_{exact} + 0.2\,\mathbb{I}_{format} + 0.3\,\mathbb{I}_{soft}$$

where $r_{exact} \in \{0, 1\}$ is the original correctness reward.

- **Reward 1: Format compliance (+0.2).** If the response includes the final-answer marker `####`, we add +0.2. This reward targets a concrete failure mode from Part Three: many generations were truncated or ended without an extractable final answer. Intuitively, it teaches the model to always finish with a parseable final answer format.
- **Reward 2: Soft match (+0.3).** If the extracted final answer is incorrect but the gold numeric answer appears anywhere in the reasoning, we add +0.3. This gives partial credit for intermediate reasoning progress and helps push near-miss cases toward exact correctness. Intuitively, it turns all-or-nothing supervision into a denser signal when the model is numerically close.

To reduce reward hacking risk, we gate soft match behind the main objective: exact match remains the dominant signal, and soft match is only applied on incorrect final answers. In addition, pairing soft match with format compliance discourages number spamming: the model is rewarded for producing a structured, complete output with a single final answer token sequence (`####`), rather than listing many candidate numbers. Empirically, this combination improves behavior (higher format compliance, lower truncation) while still increasing exact GSM8K accuracy.

All runs used the same optimizer, rollout setup, and training schedule as the original RL script by Karpathy [5]; only reward logic changed for GSM8K [6].

4.2 Reward System Performance

We evaluate four RL runs: **Run #1** baseline (original reward only), **Run #2** combined rewards (format + soft match), **Run #3** format-only reward, and **Run #4** soft-match-only reward.

Run	Acc.	Format	Trunc.	Near Miss	Avg Steps	Avg Chars
Baseline	12.5%	74.5%	22.5%	11	2.38	306.6
Combined (+F,+S)	**14.0%**	**91.5%**	**5.5%**	**15**	**2.70**	**339.6**
Format Only (+F)	13.0%	89.5%	8.0%	12	2.52	327.8
Soft Match Only (+S)	11.0%	66.5%	25.5%	12	2.44	313.5

Table 5: Part Four RL results under identical training configuration with different reward systems.

Metric note: Near Miss counts predictions within 10% of the gold numeric answer; Avg Steps is the average detected reasoning-step count; Avg Chars is the average response length in characters.

Run #2 (combined rewards) is the strongest overall setup: it gives the best GSM8K accuracy (14.0%) and the largest drop in truncation (22.5% → 5.5%). Run #3 (format-only) is the strongest single-reward ablation and already improves over baseline. Run #4 (soft-match-only) underperforms baseline, showing that soft match without output-structure pressure is not sufficient.

4.3 Mistake Comparison and Visualizations

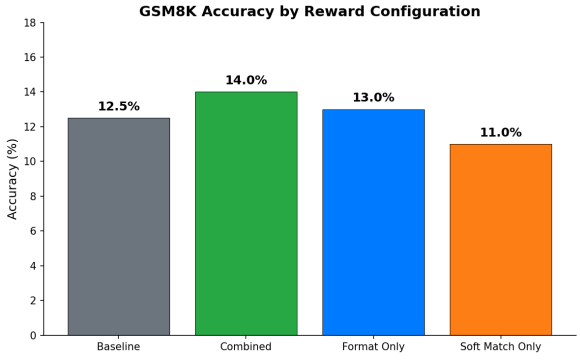

Figure 6: GSM8K accuracy across reward configurations.

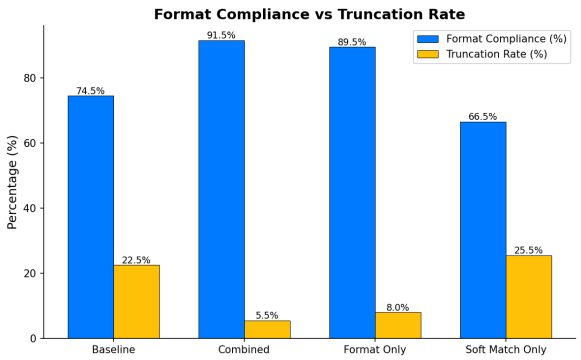

Figure 7: Format compliance vs truncation rate.

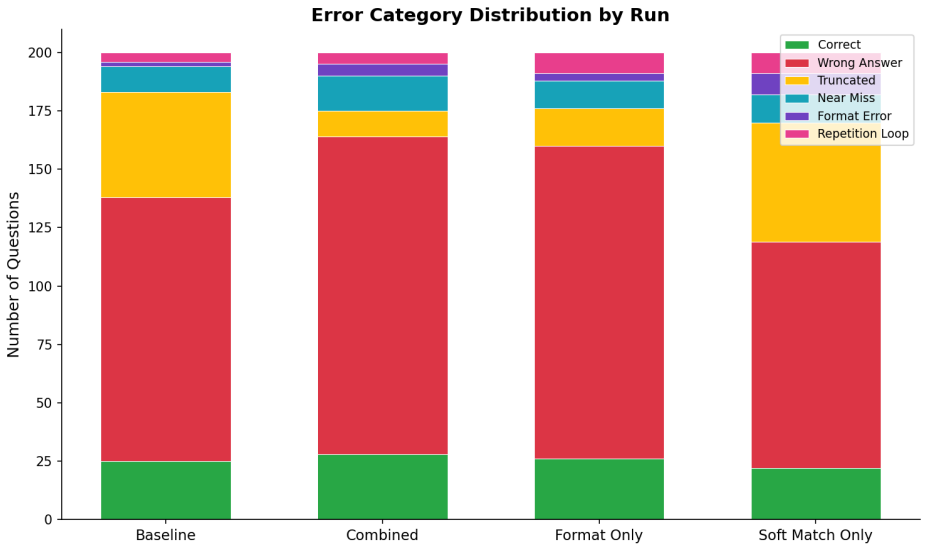

Figure 8: Error-category counts for baseline, combined, and isolated-reward runs.

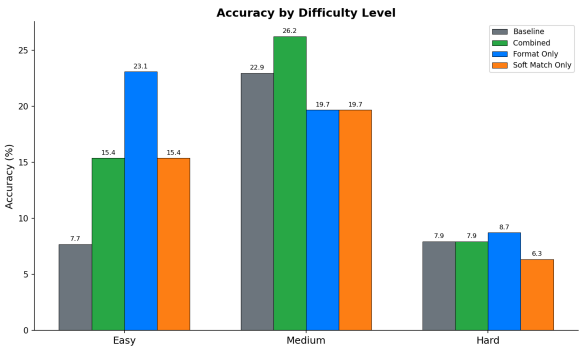

Figure 9: Accuracy by GSM8K difficulty split.

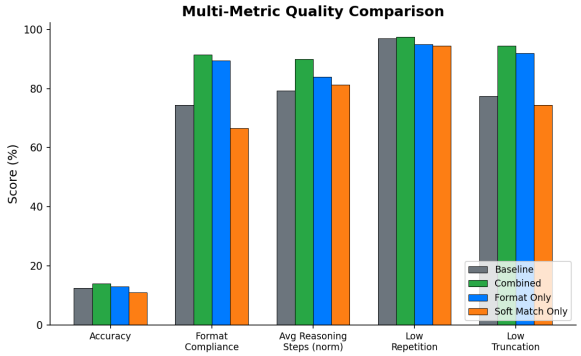

Figure 10: Multi-metric quality comparison.

Mistake-Type Differences

Compared with Run #1 baseline, Run #2 combined converts many failures from *truncated* outputs into complete answers. Truncations drop from 45 to 11 examples, while wrong-answer counts rise from 113 to 136 because the model now finishes more responses and commits to an answer. Near misses increase from 11 to 15, indicating better partial reasoning even when exact arithmetic is still wrong.

Separate Environments: Run #3 and Run #4

When rewards are isolated in separate environments, the format-only run (Run #3) remains robust: 13.0% accuracy, 89.5% format compliance, and 8.0% truncation. In contrast, the soft-match-only run (Run #4) degrades to 11.0% accuracy and 25.5% truncation. This comparison against Run #1 and Run #2 shows that format compliance is the key stabilizing signal, while soft match works best as a secondary shaping term once structured outputs are enforced.

Impact on Error Behavior

Figure 8 and Figure 7 show the clearest behavioral change: reward shaping strongly affects *how* the model fails. Combined rewards reduce collapse/truncation behavior and improve completion quality, but do not fully solve hard multi-step arithmetic; hard-split accuracy remains roughly flat across runs (about 6–9%), consistent with model-capacity limits.

4.4 Summary of Results

Experiment	**Key Metric**	**Impact Commentary**
SFT (Original)	GSM8K: 6.29%	Adds instruction following but still weak on multi-step arithmetic reasoning.
RL Run #1 (Original)	GSM8K: 12.5%	Large jump over SFT; however truncation remains high (22.5%).
RL Run #3 (+Format Only)	GSM8K: 13.0%	Best single reward; strongly improves completion behavior and reduces truncation to 8.0%.
RL Run #4 (+Soft Match Only)	GSM8K: 11.0%	Underperforms baseline when used alone; encourages numeric overlap without enough output structure.
RL Run #2 (+Format + Soft Match)	GSM8K: **14.0%**	Best overall run; highest accuracy, highest format compliance (91.5%), and lowest truncation (5.5%).

Table 6: End-to-end ablation summary for Part Four, including isolated and combined reward systems.

References

[1] Longhui Yu et al. MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models. *arXiv preprint arXiv:2309.12284*, 2023.

[2] Yuxiang Wei et al. Magicoder: Source Code Is All You Need. *arXiv preprint arXiv:2312.02120*, 2023.

[3] Luyu Gao et al. PAL: Program-Aided Language Models. *arXiv preprint arXiv:2211.10435*, 2023.

[4] Zhihong Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.

[5] Andrej Karpathy. Nanochat repository. *GitHub*, 2025.

[6] Karl Cobbe et al. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*, 2021.